# Netherlands

# Bicycles

- More bicycles than people
- 53% of Dutch people use their bikes to commute
- Unique infrastructure just for bicycles
- Largest bicycle parking in the world (13.500 spots)


# Bicycles


# King's day

- Celebrate the birthday of our king, Willem Alexander (prins pils)
- Morning free market
- Partying and drinking beer



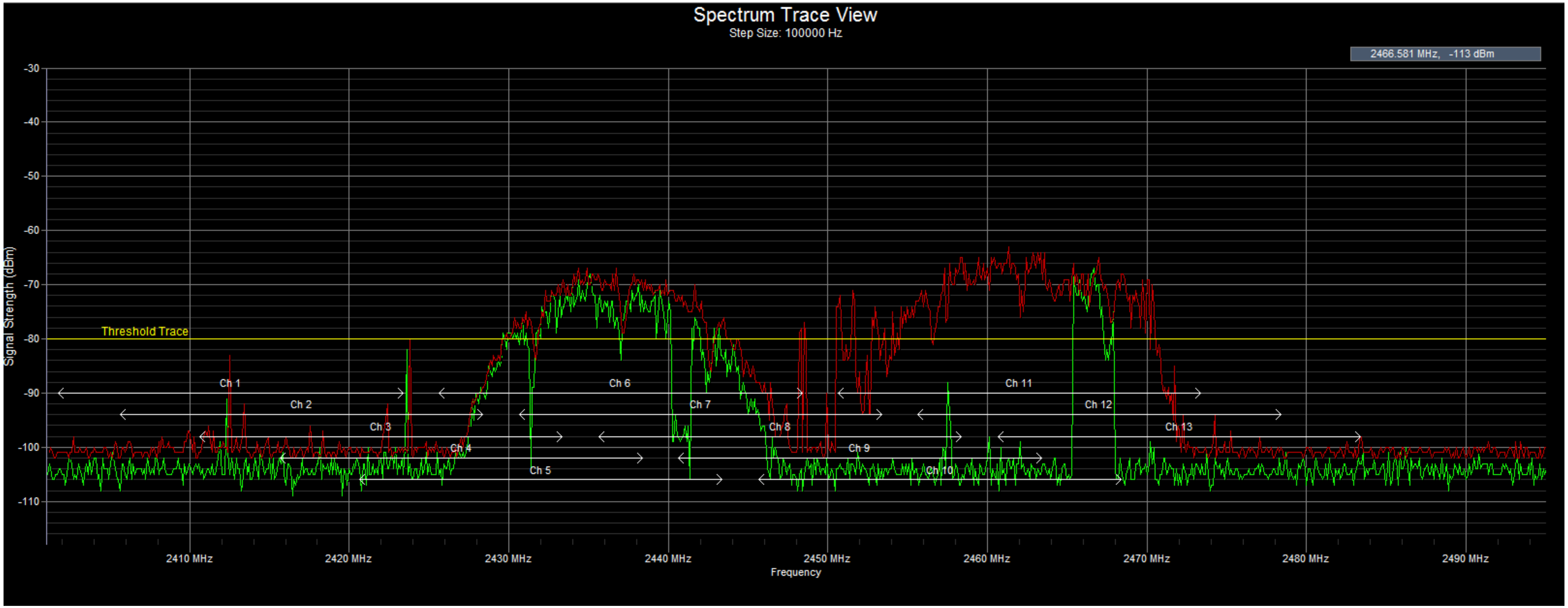
KINGSLAND

WOK

Laat Me!

# Spectrum Sensing for Cognitive Radio Applications

By: Lars Vonk

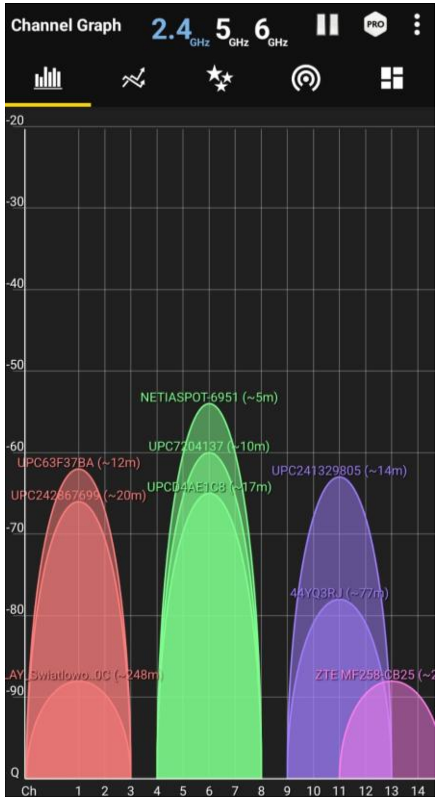


# Presentation outline

- Problem statement
- Context
- Literature
- Methodology
- Results
- Conclusion

# Problem statement

- Wireless communication
- Many users
- Spectrum shortage

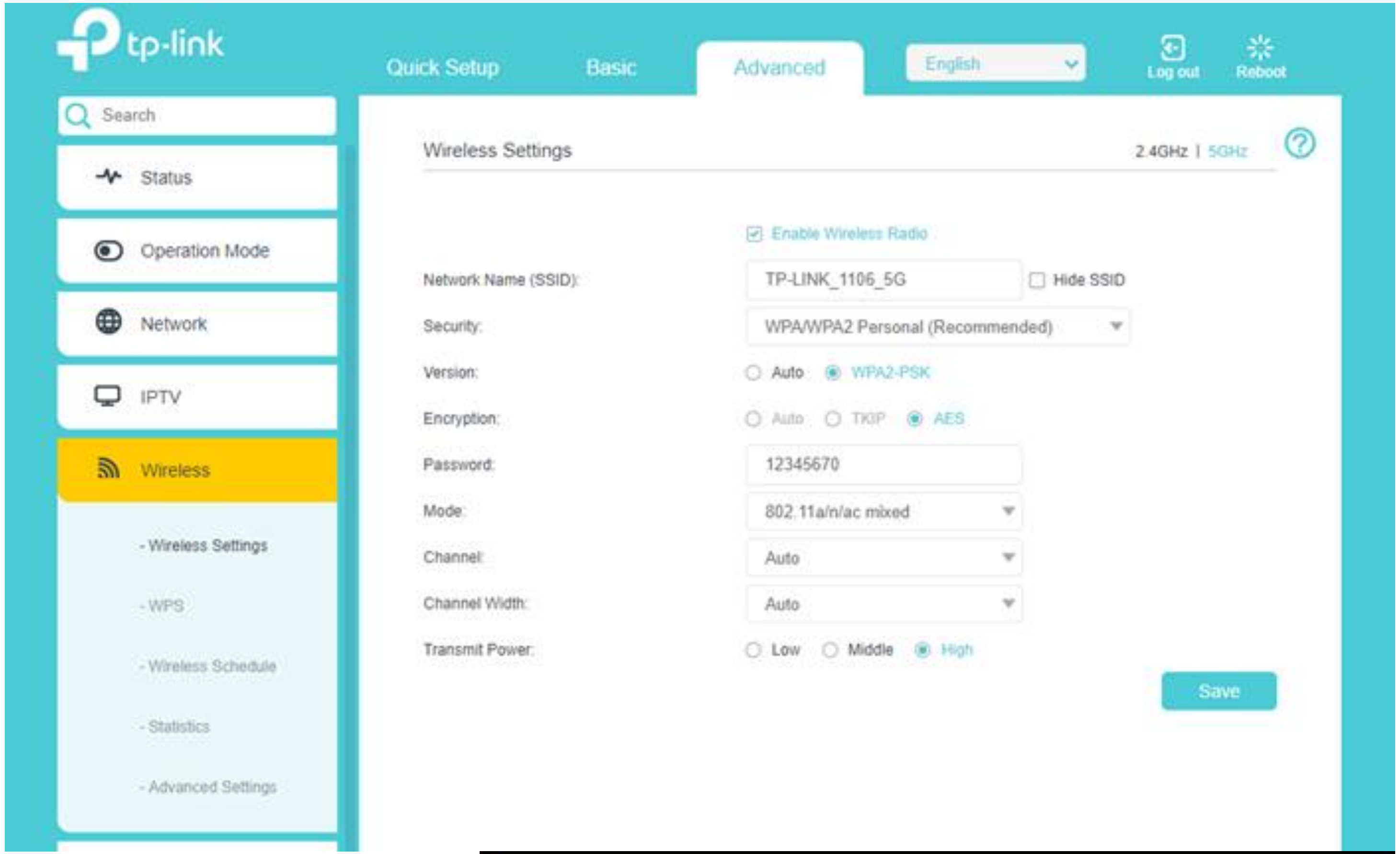


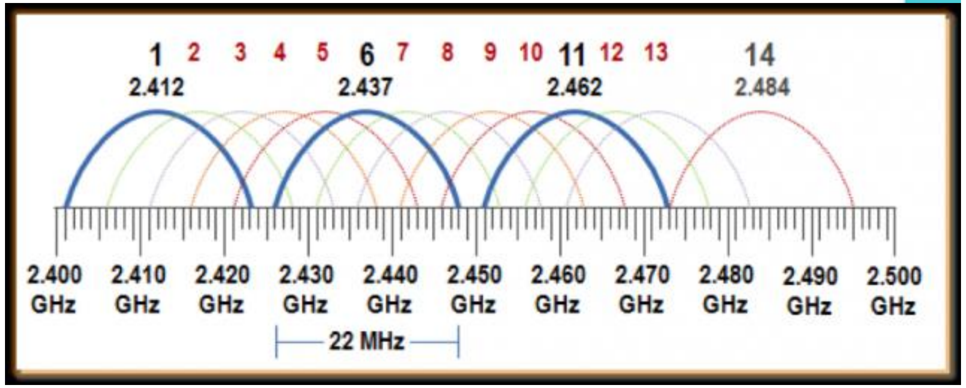

Spectrum at home

Image from paper: Street-Scale Mapping of Urban Radio Frequency Noise at Very High Frequency and Ultra High Frequency

# Context (1)

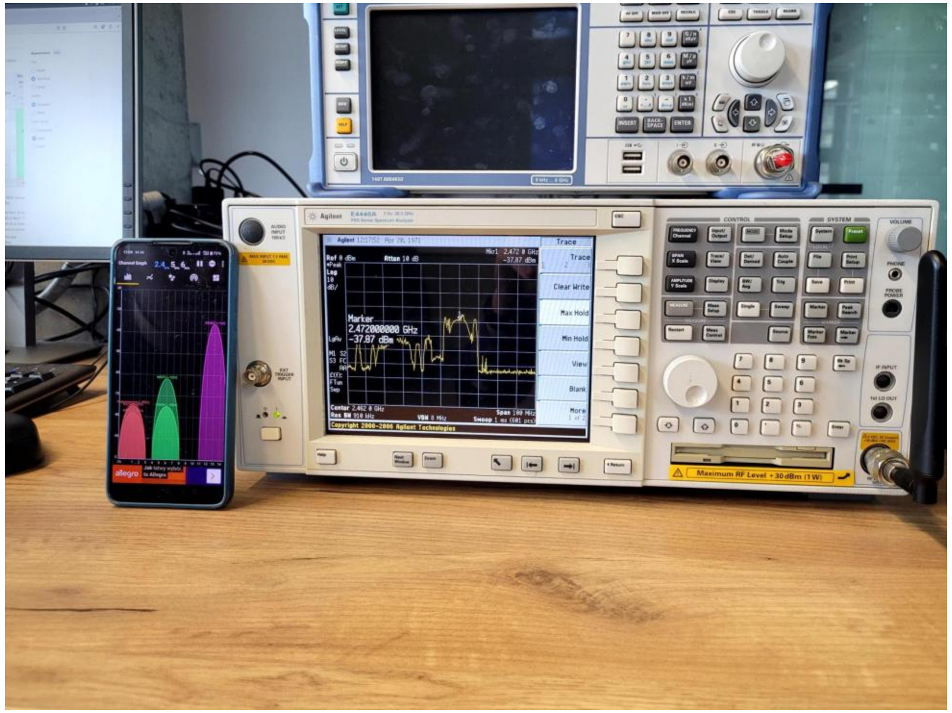
- Limited channels
- Channel selection
- Urban high population areas
- Well-researched topic

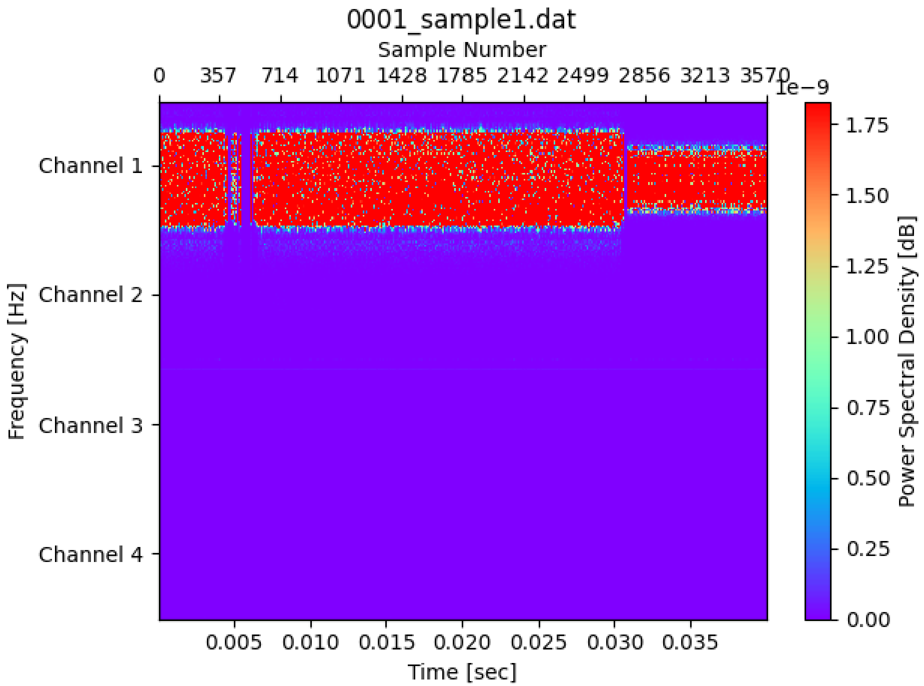


# Context (2)

- Training data easily available





# Context (3)

## Data representation

0.849 0.161 0.025 0.101 0.393 0.818 0.064 0.521 0.286 0.904 0.141 0.826 0.077 0.609 0.336 0.406 0.498 0.189 0.422 0.440 0.446 0.346 0.071 0.182 0.690 0.451 0.412 0.247 0.941
0.567 0.202 0.417 0.347 0.271 0.901 0.322 0.750 0.537 0.520 0.350 0.895 0.995 0.042 0.633 0.650 0.929 0.161 0.161 0.885 0.671 0.254 0.814 0.294 0.408 0.922 0.735 0.932 0.934

- Time based IQ samples
- Using the Fourier transformation for analysis (time domain to frequency domain)

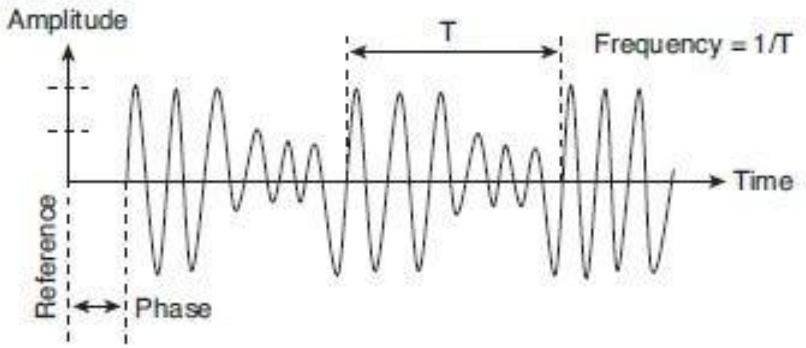
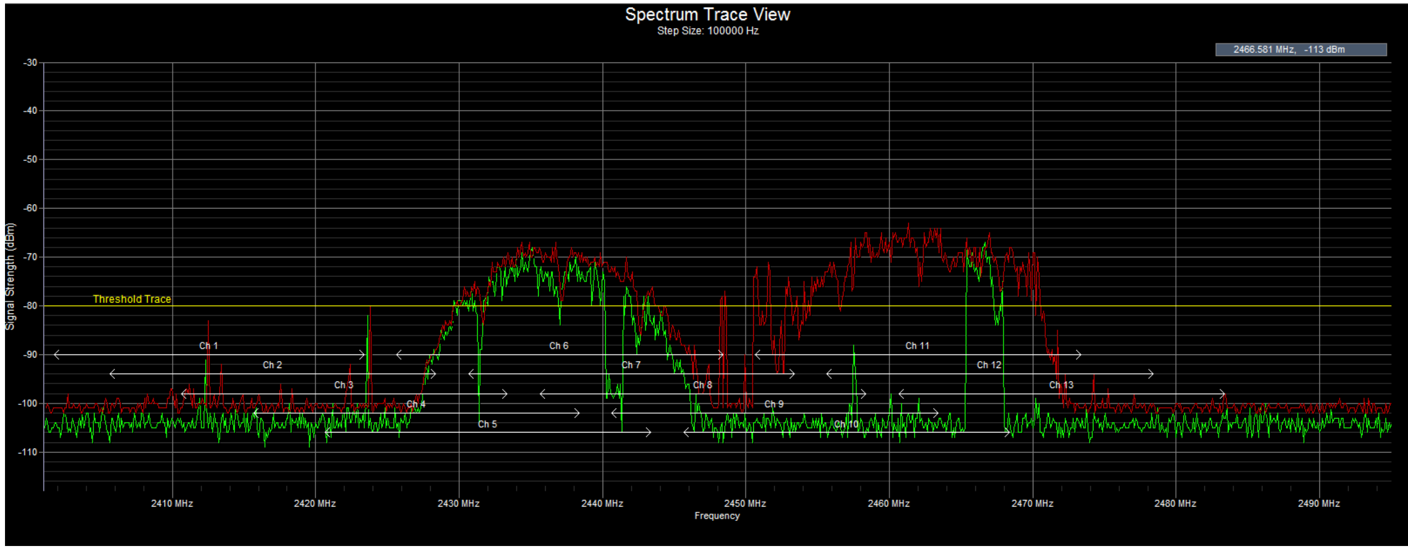
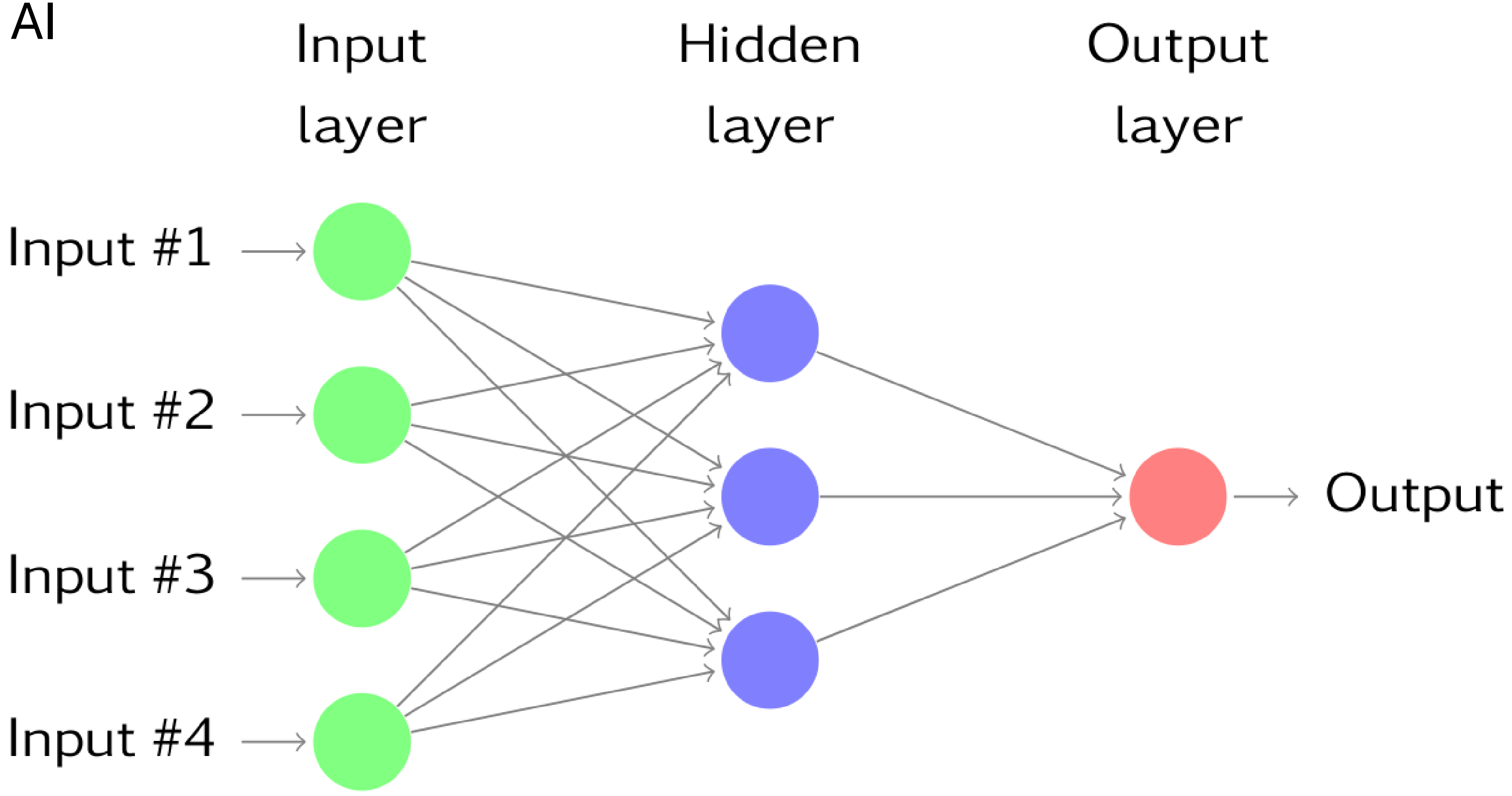



**Figure 2-1** *The Amplitude, Frequency, and Phase Elements of a Radio Wave*



# Context (4)

- Spectrum data contains patterns
- Classification problem
- Good use case for AI

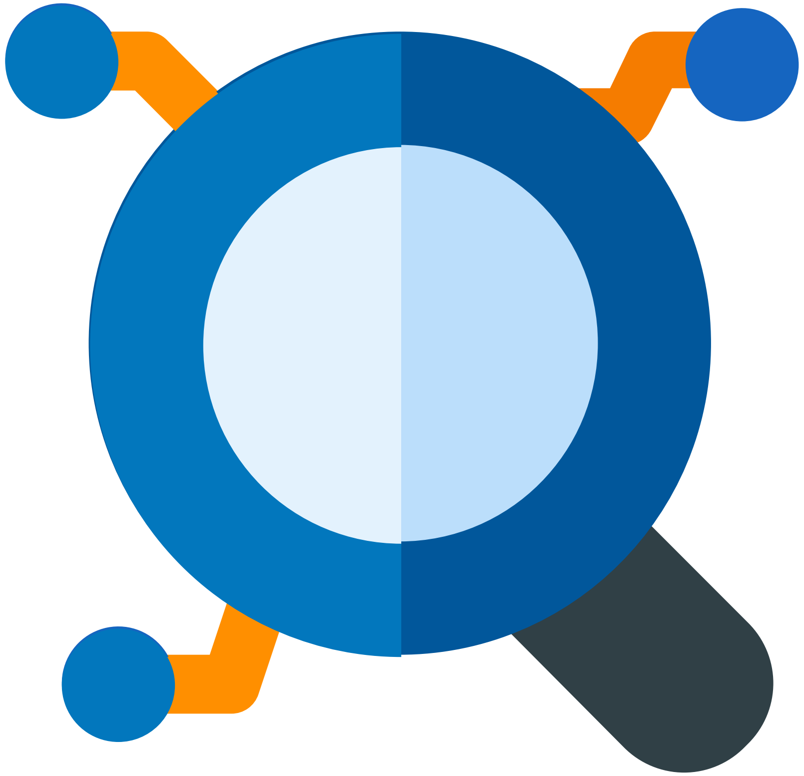


# Literature

- Well defined problem
- Good results with Neural Networks
- Confirmation bias (single model)
  - SVM
  - Convolutional neural network
  - Long Short-term memory model (prediction)

# Methodology

- Tools I’ve used
- Creating input data
- Testing all the models
- Combining and improving
- Results

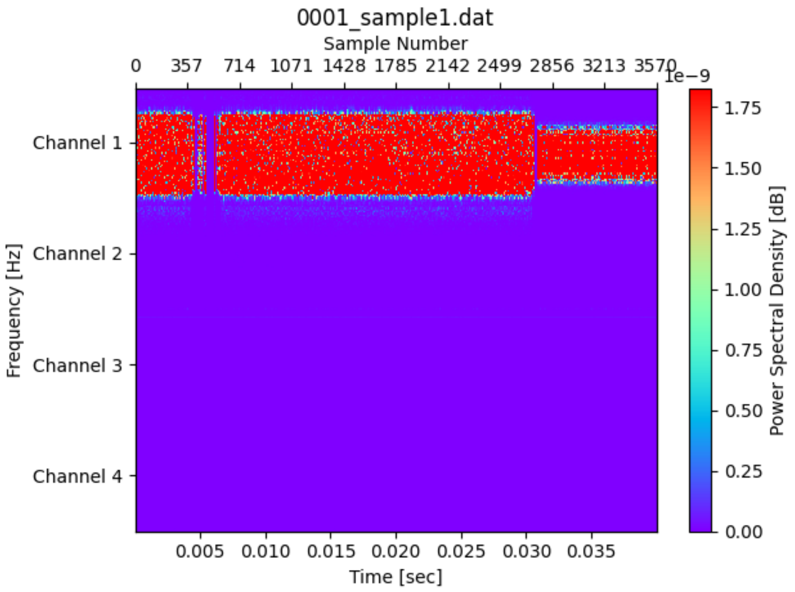


# Methodology: Tools I’ve used

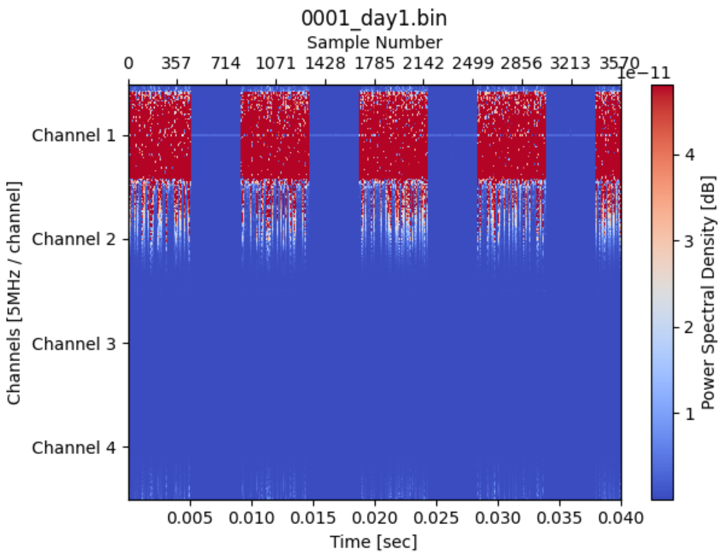
- PyTorch (model creation)
- NumPy (data manipulation)
- Matplotlib + Scikit (data analysis)

# Methodology: Creating input data

## Generating dataset



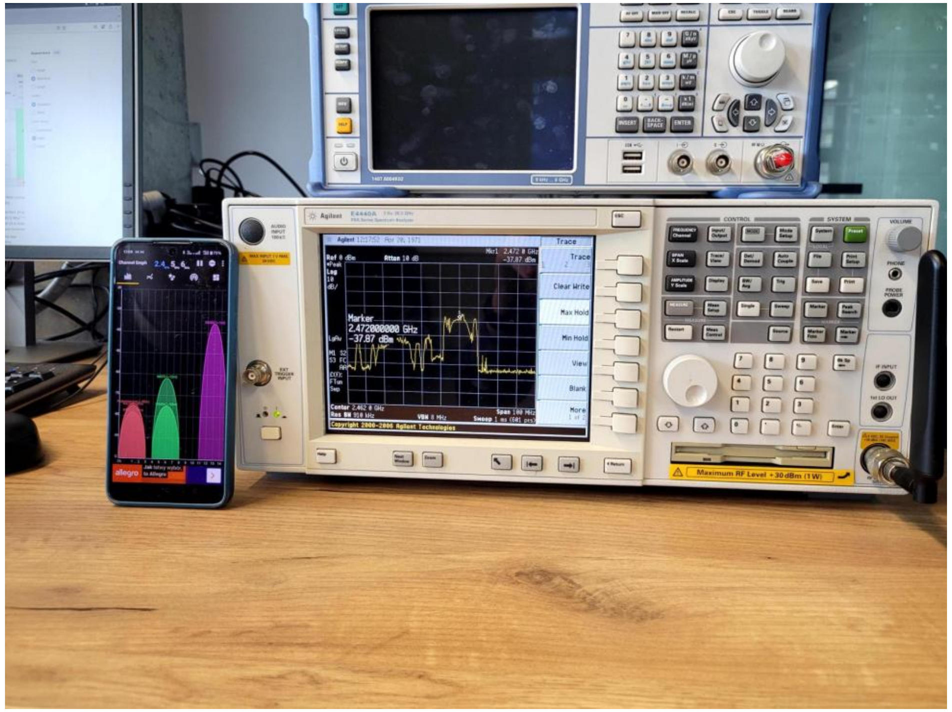

Recorded data
(lab experiment)



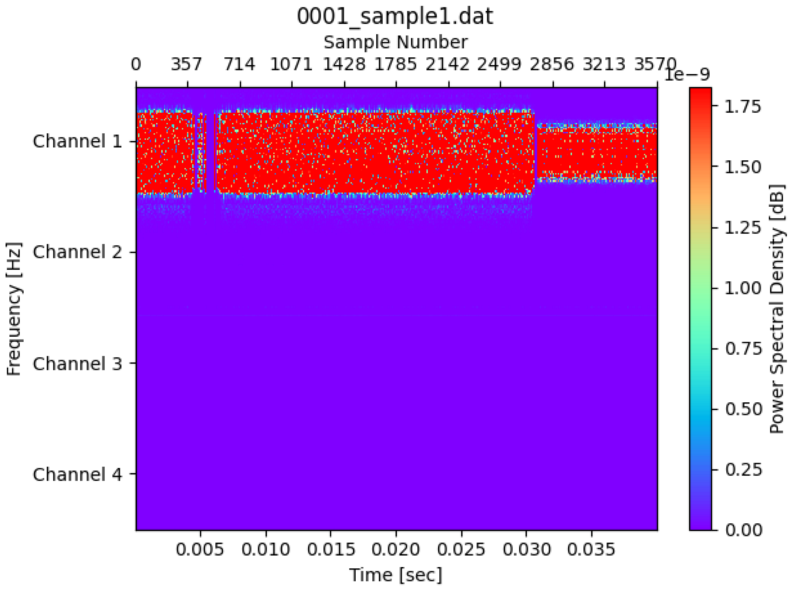

External data
(research paper)

# Methodology: Creating input data

- Manually choosing channel
- Download of big file

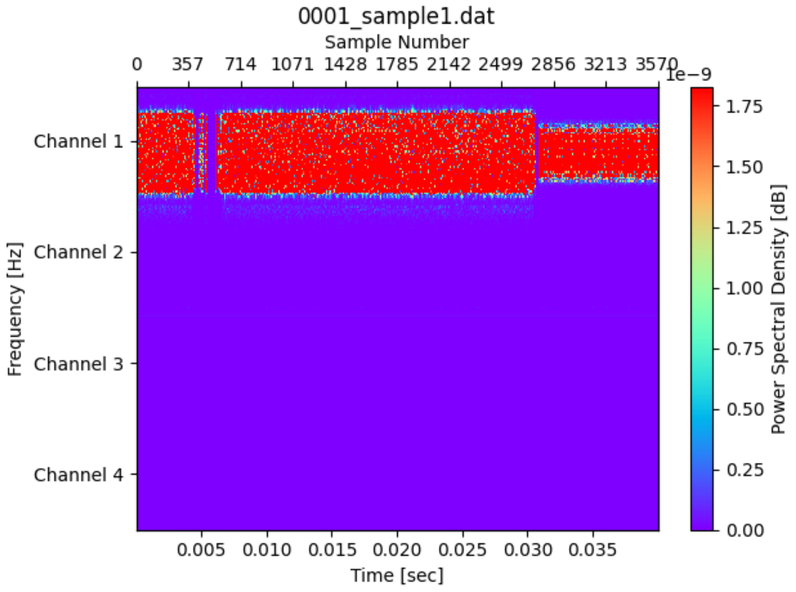


Spectrum analyser

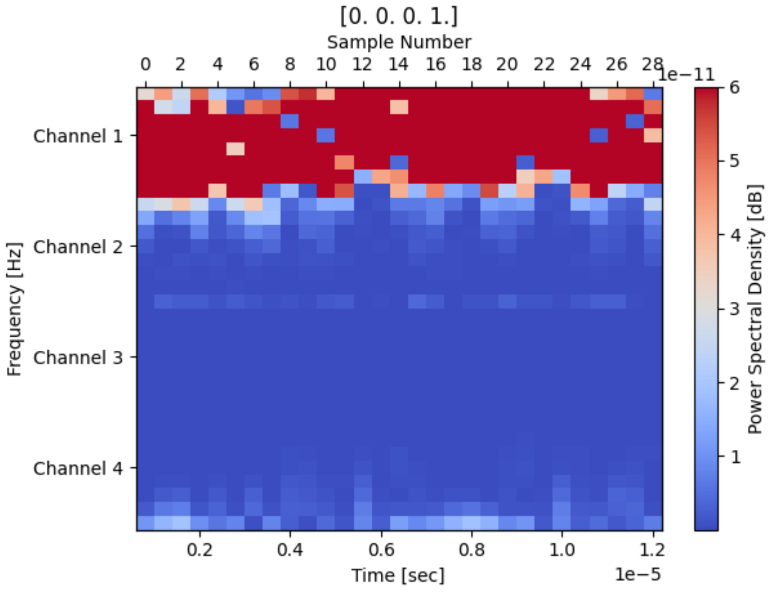


# Methodology: Creating input data

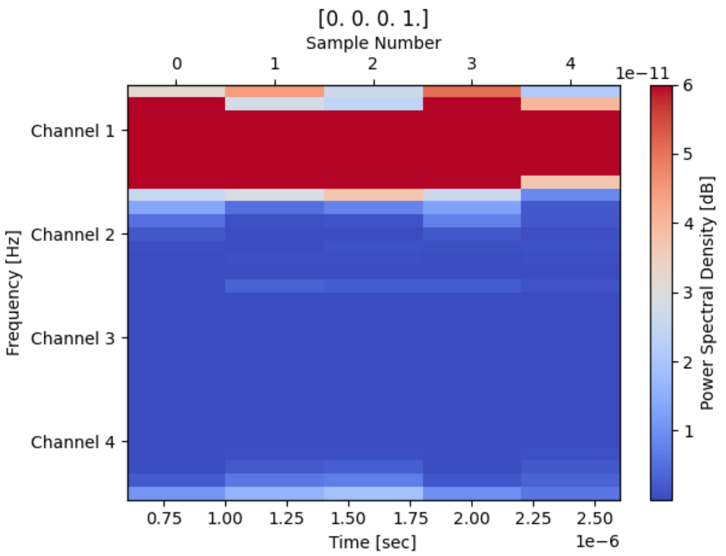
## Creating sequences




Full dataset



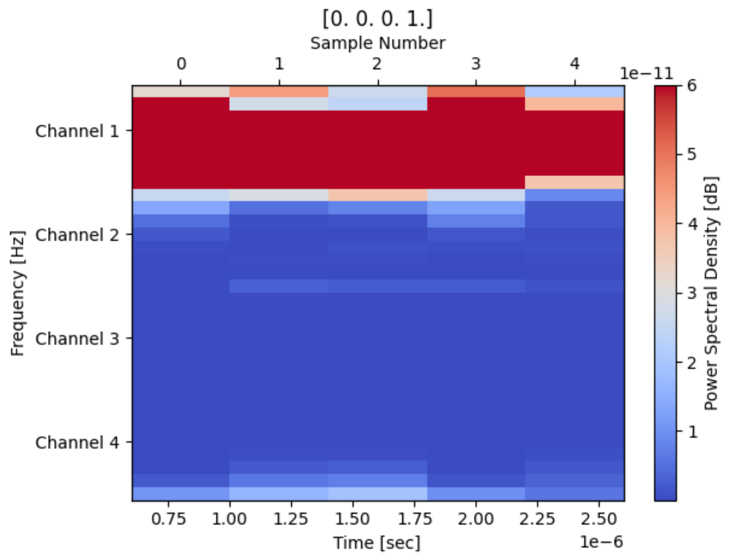

256 sample sequence




32 sample sequence

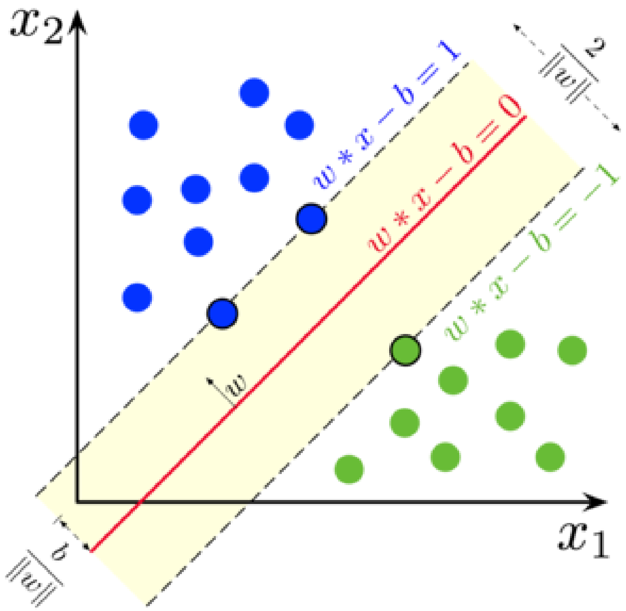
# Methodology: Creating input data

## Representing a sequence

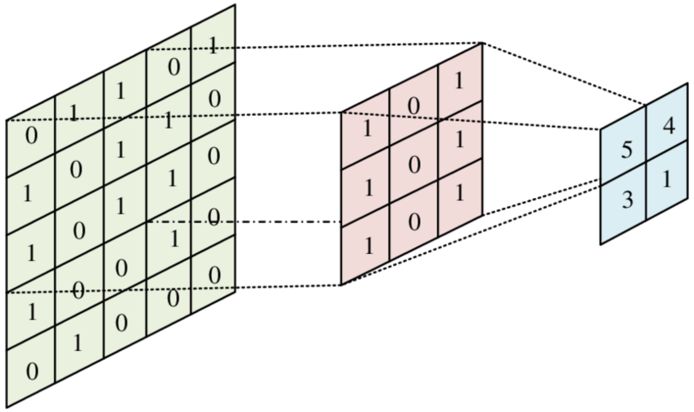



| Sample number | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | ... | 32 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Real number | 0.35 | 0.24 | 0.16 | -0.52 | 0.78 | -0.26 | 0.45 | 0.18 | -0.12 | -0.84 | 0.82 | 0.77 | 0.11 | -0.89 | -0.88 | ... | -0.16 |
| Imaginary number | -0.12 | -0.84 | 0.82 | 0.77 | 0.11 | -0.89 | -0.88 | -0.16 | 0.35 | 0.24 | 0.16 | -0.52 | 0.78 | -0.26 | 0.45 | ... | 0.18 |

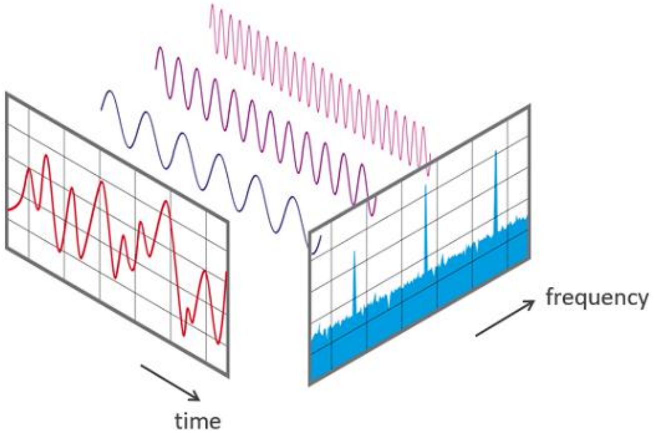
# Methodology: Testing all the models




Support vector machine



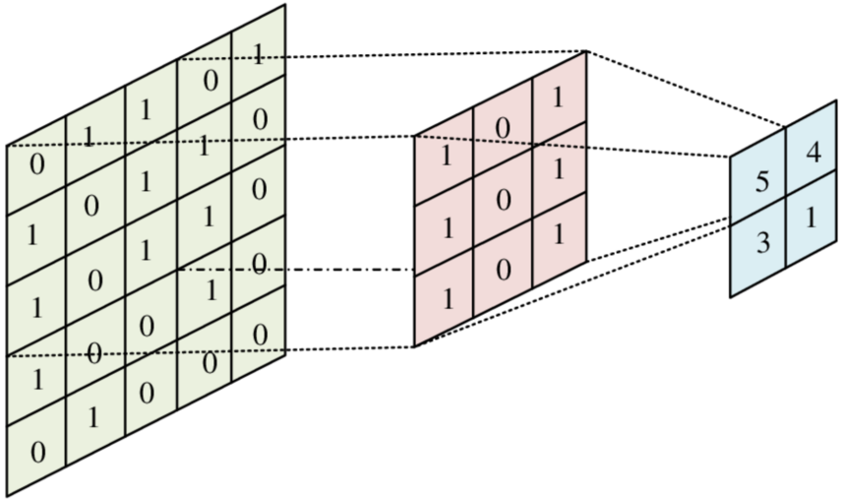
Convolutional neural network

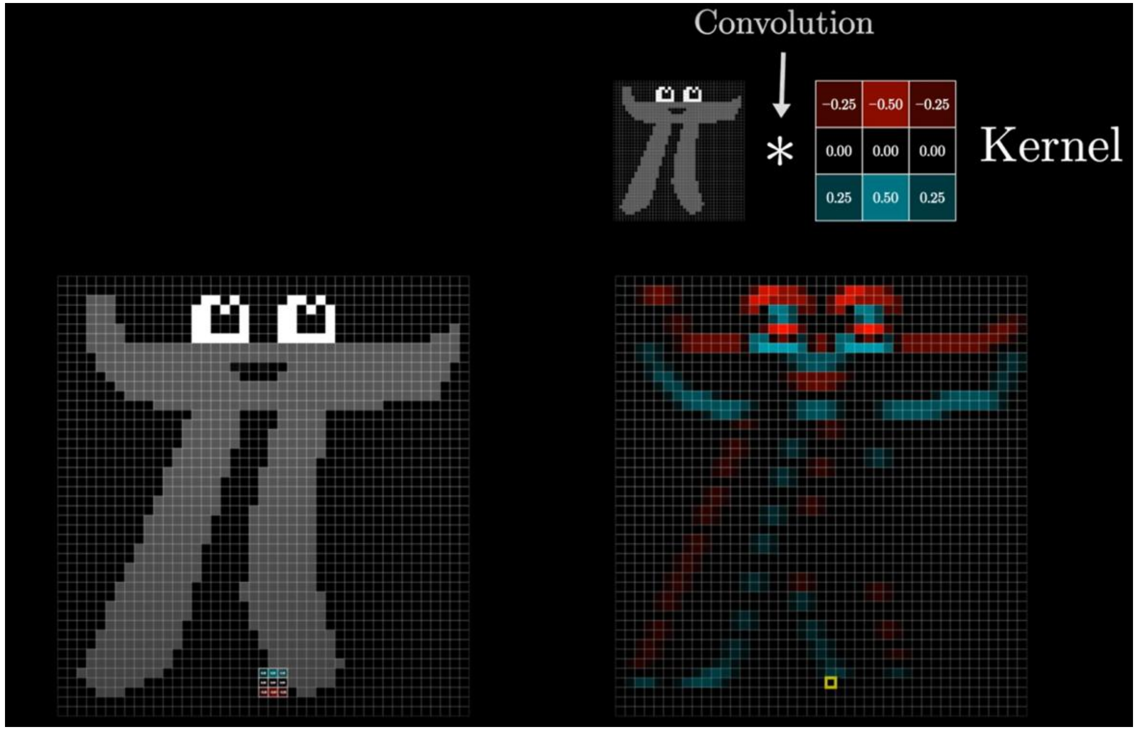



Fast Fourier transform

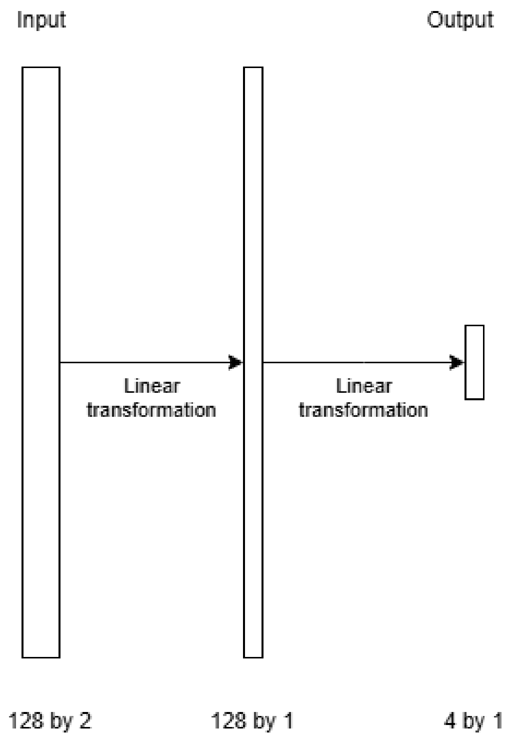
# Short note: What is convolution?

- Initially invented for image processing

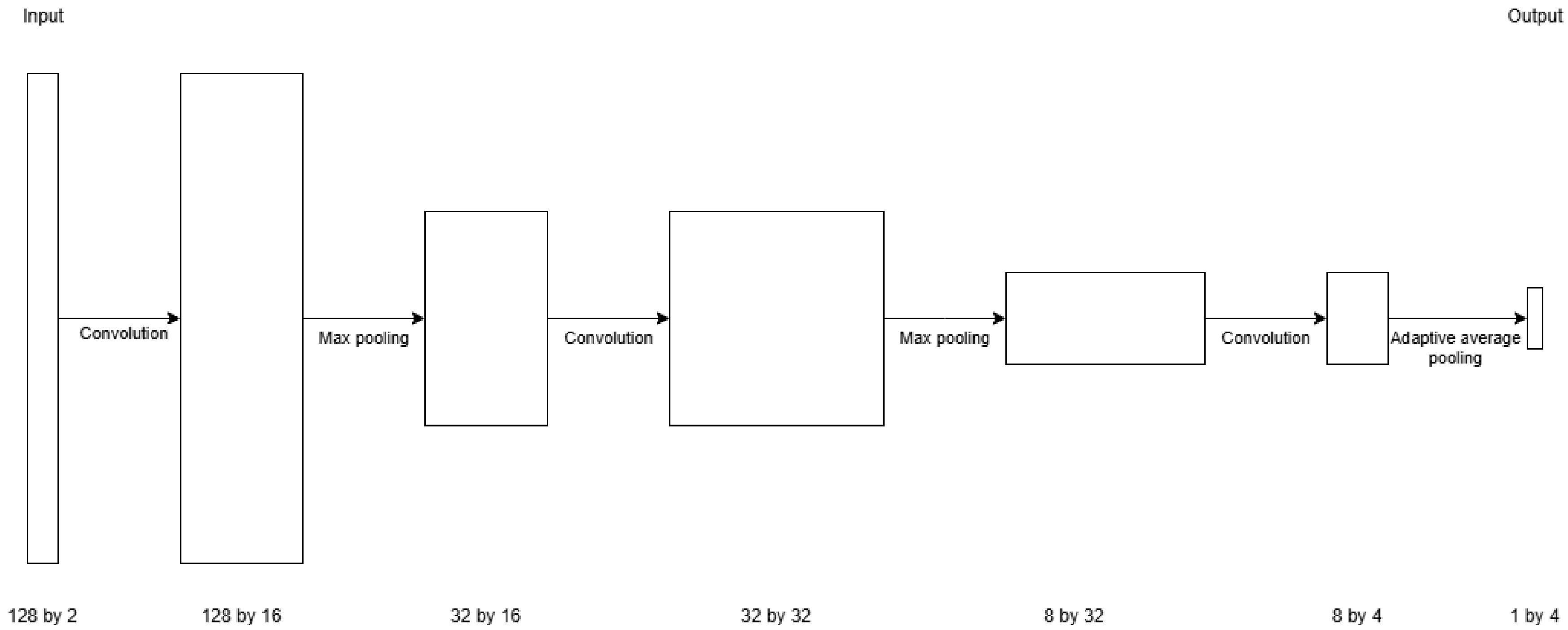


# Methodology: Testing all the models




Support Vector Machine

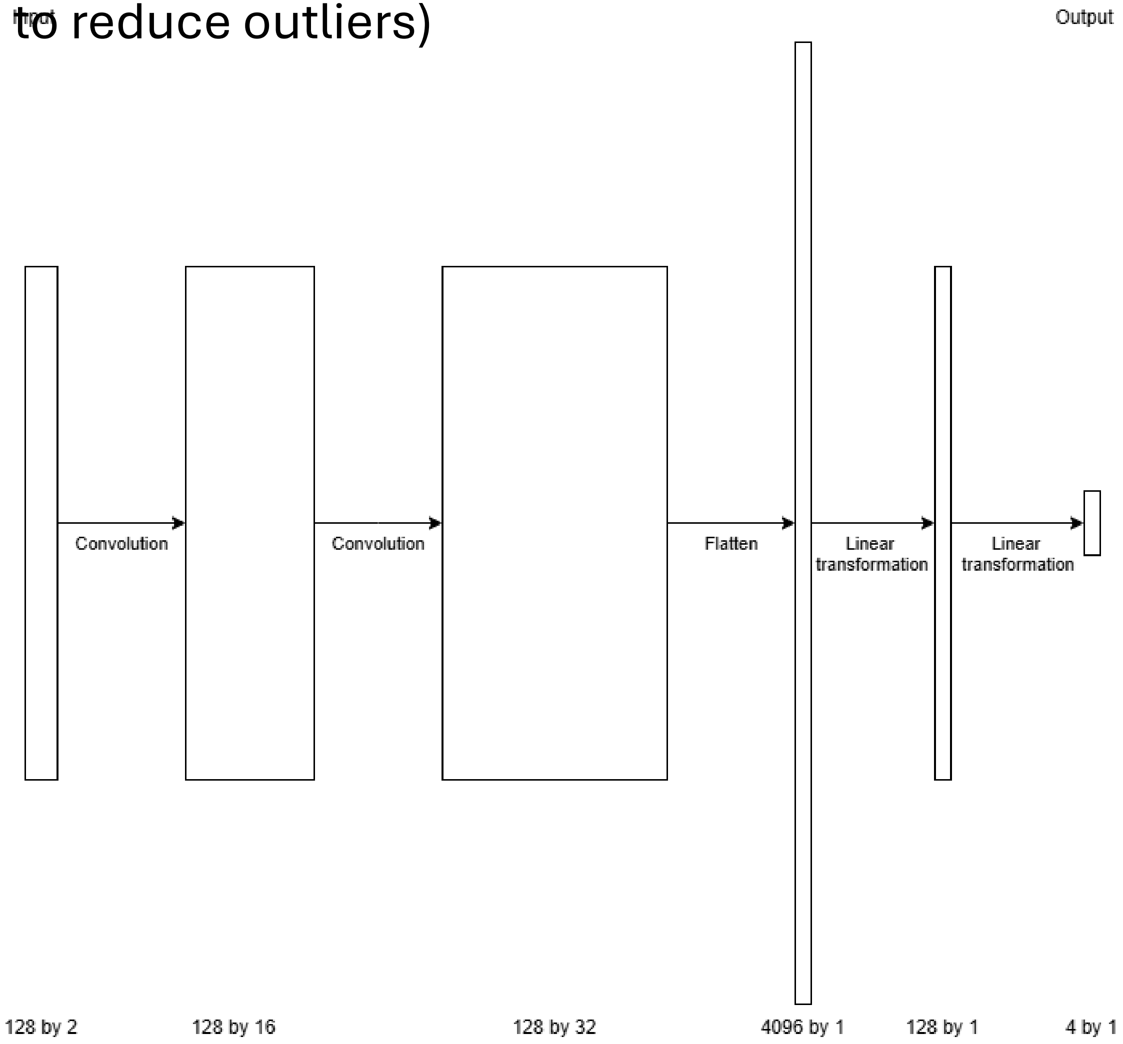



Convolutional model

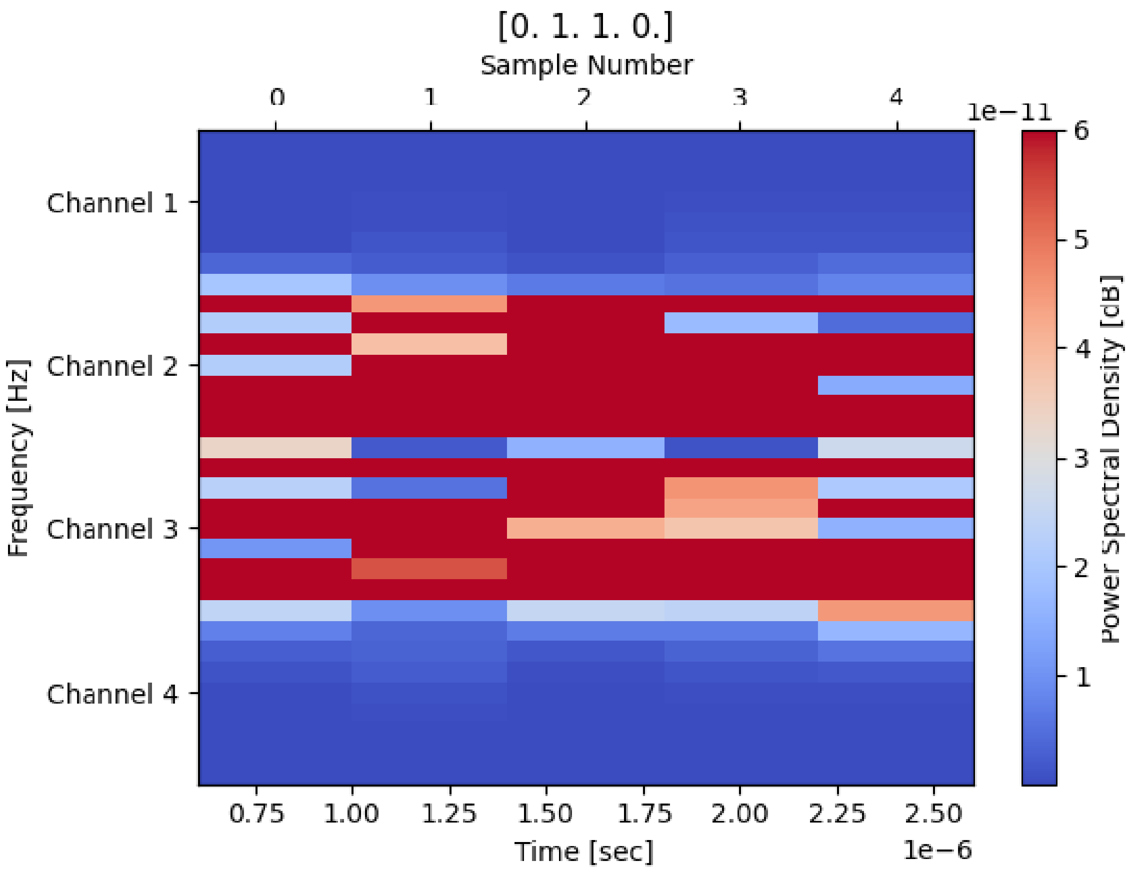
# Methodology: Combining and improving

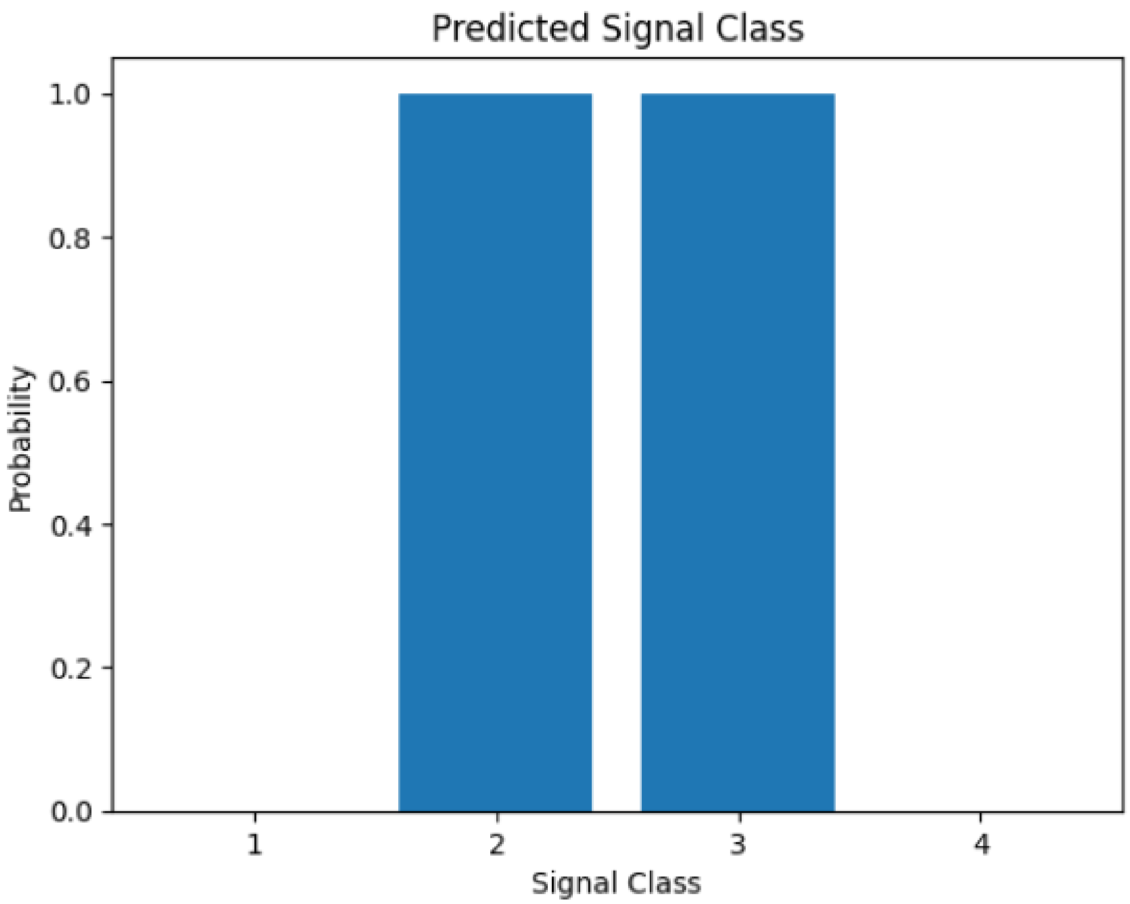
- Convolutional layer + Support Vector Machine
- Adding batched normalisation (normalising values to reduce outliers)

Convolutional Support
Vector Machine



# Results


[0. 1. 1. 0.]
Sample Number
0
1
2
3
4
1e−11
Channel 1
Channel 2
Channel 3
Channel 4
Frequency [Hz]
6
5
4
3
2
1
Power Spectral Density [dB]
0.75
1.00
1.25
1.50
1.75
2.00
2.25
2.50
Time [sec]
1e−6

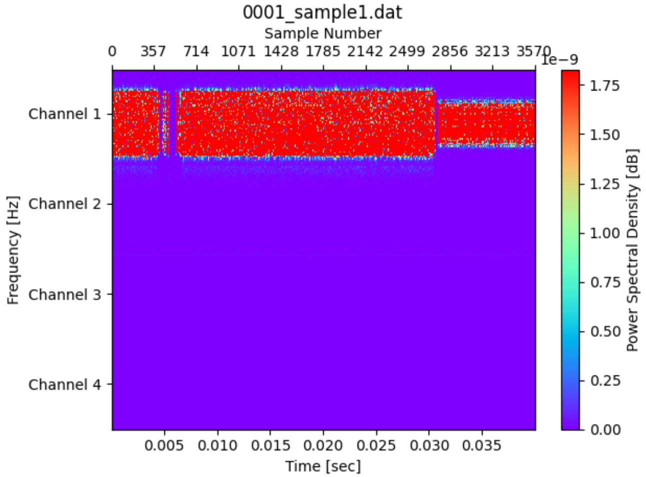

Predicted Signal Class
1.0
0.8
0.6
0.4
0.2
0.0
Probability
1
2
3
4
Signal Class

# Results

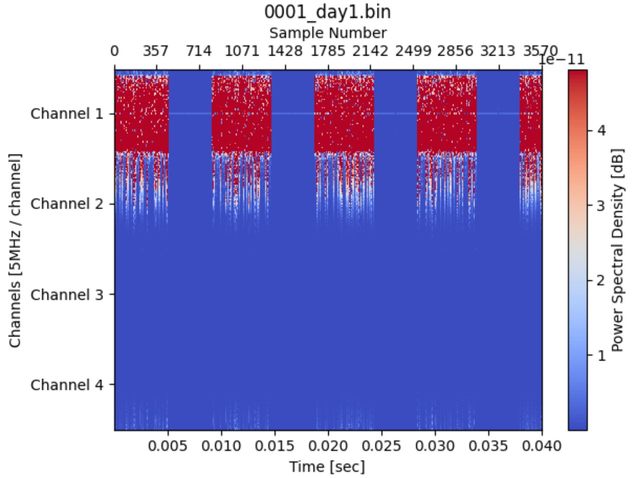
| Sequence length | 32 | | 64 | |
|---|---|---|---|---|
| Sampling divisor | 1 | 2 | 1 | 2 |
| SVM | 93.06% | 71.14% | 94.19% | 73.22% |
| Conv. | 92.06% | 81.18% | 94.88% | 67.33% |
| Conv. SVM | 97.80% | 86.03% | 98.83% | 77.86% |
| Conv. SVM w/ batched norm. | 99.10% | 89.47% | 99.65% | 91.93% |
| FFT | 86.01% | 74.54% | 91.28% | 73.90% |

| Sequence length | 128 | | 256 | | |
|---|---|---|---|---|---|
| Sampling divisor | 1 | 2 | 1 | 2 | 4 |
| SVM | 90.02% | 75.06% | 90.45% | 75.38% | 62.80% |
| Conv. | 97.73% | 84.47% | 97.19% | 86.56% | 72.49% |
| Conv. SVM | 99.76% | 92.04% | 99.68% | 92.53% | 76.99% |
| Conv. w/ batched norm. | 99.33% | 93.81% | 99.89% | 95.89% | 70.79% |
| FFT | 96.95% | 84.37% | 97.76% | 87.32% | 83.22% |

# Results



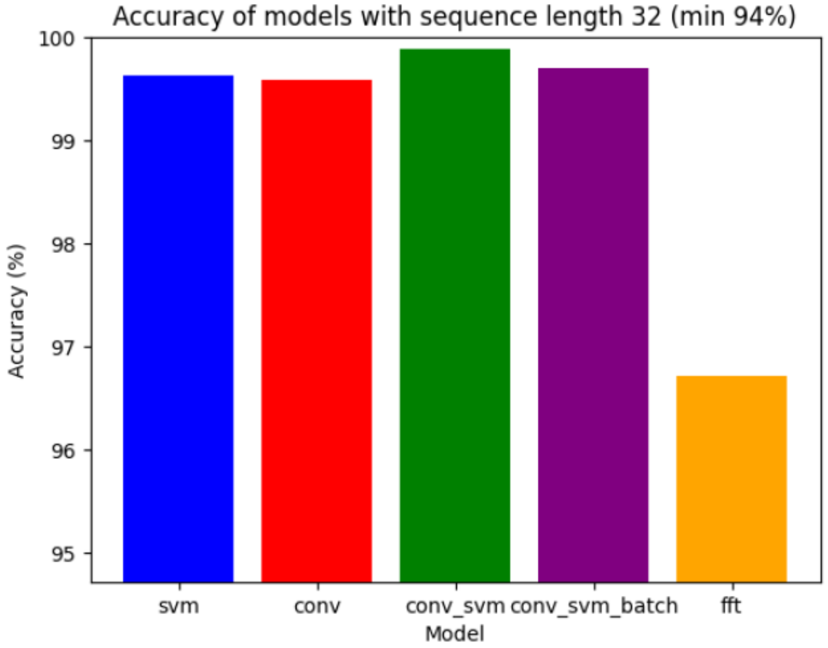

Recorded




External

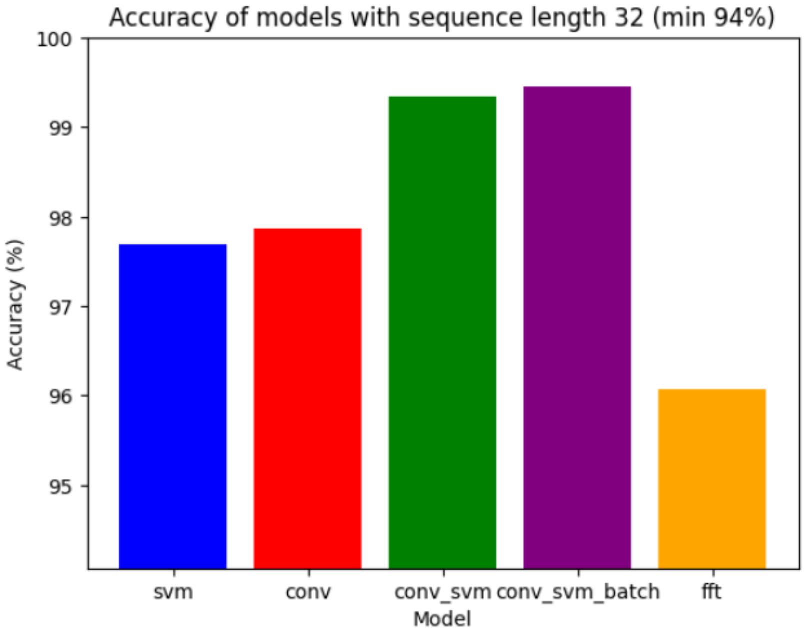
| **Sequence length** | **32** | | | |
|---|---|---|---|---|
| **Sampling divisor** | **1** | | **2** | |
| **Dataset source** | rec. | ext. | rec. | ext. |
| SVM | 99.62% | 93.06% | 97.68% | 71.14% |
| Conv. | 99.58% | 92.06% | 97.87% | 81.18% |
| Conv. SVM | 99.88% | 97.80% | 99.34% | 86.03% |
| Conv. SVM w/ batched norm. | 99.70% | 99.10% | 99.46% | 89.47% |
| FFT | 96.72% | 86.10% | 96.07% | 74.54% |

# Results



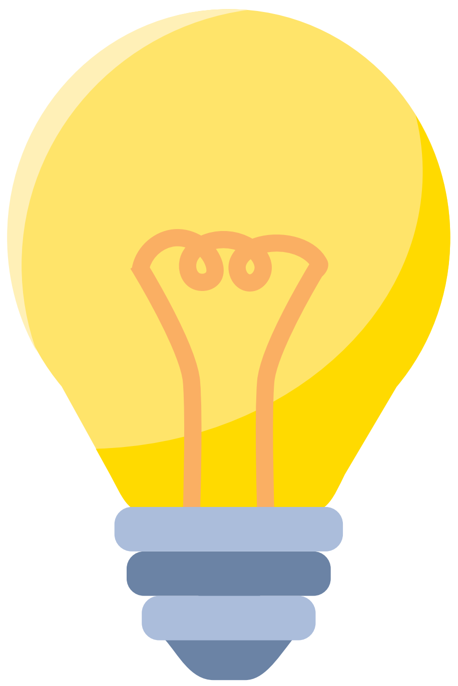

(a) sequence length of 32

Full sampling rate



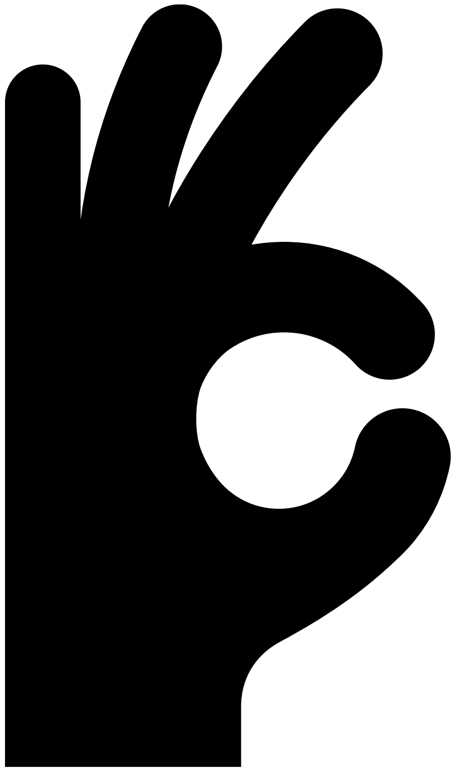

(a) sequence length of 32

Halved sampling rate



# Conclusion

- Simple Neural networks can find frequency-based patterns in time-based data
- The model is more resilient than using FFT in high noise environments
- A combination of previous research using a convolutional layer which output is used by a Support Vector Machine produces improved results



Thank you